Date	01 October 2026
Team ID	SWTID-2026-6912
Project Name	FitBuddy - AI Fitness Plan Generator using Gemini Models
Maximum Marks	5 Marks

Code-Layout, Readability and Reusability

This document evaluates the quality of the FitBuddy codebase in terms of its structure, readability, and reusability. The assessment is based on the project architecture and modules described in the submitted project document. FitBuddy uses FastAPI, Gemini models, SQLAlchemy/SQLite, and Jinja2 templates, with functionality divided into reusable modules.

Code Layout Checklist

S.No	Code Quality Parameter	Description	Followed (Yes / No / Partial)	Remarks
1	Consistent Indentation	Uniform spacing/tabs used throughout the code	Yes	Code is organized across Python modules and HTML templates.
2	Proper File Structure	Files and folders are logically organized	Yes	The project separates app code, templates, static assets and supporting modules.
3	Meaningful Variable Names	Variables reflect their purpose clearly	Yes	User, goal, intensity, workout_plan and nutrition_tip are descriptive.
4	Function / Method Names	Functions are descriptively named	Yes	Examples include generate_workout_gemini(), update_workout_plan(), save_user().
5	Code Comments	Inline and block comments explain logic	Partial	The modular functions and documentation explain major logic; more inline comments could improve maintainability.
6	Modular Design	Code is split into reusable functions/modules	Yes	Gemini generation, database operations and route logic are separated into modules.
7	No Redundant Code	Duplicate or unused code is removed	Yes	Core responsibilities are separated into dedicated functions and files.
8	Error Handling	Exceptions and errors are handled gracefully	Partial	Input validation is supported with Pydantic; additional explicit exception handling can further strengthen robustness.

Reusable Components / Modules

S.No	Component / Module Name	Language / Technology	Where Reused	Reusability Level (High / Medium / Low)
1	Gemini workout generation	Python / Gemini 1.5 Pro	Workout generation and feedback-based plan updates	High
2	Gemini nutrition tip generation	Python / Gemini Flash	Nutrition/recovery tip generation for user goals	High
3	Database functions	Python / SQLAlchemy / SQLite	Saving users, plans and updating revised plans	High

S.No	Component / Module Name	Language / Technology	Where Reused	Reusability Level (High / Medium / Low)
4	FastAPI route handlers	Python / FastAPI	Connect frontend inputs, AI modules and database operations	High
5	Jinja2 templates	HTML / CSS / Jinja2	Dynamic rendering of index, result and admin pages	High

Team Work Allocation

S.No	Team Member	Assigned Work / Milestone	Key Responsibilities
1	Pavithra M	Milestone 1 – Model Selection & Architecture	Research/select Gemini models, define FastAPI architecture, and set up the development environment.
2	Manisha S	Milestone 2 – Core Functionalities	Develop workout generation, nutrition tips, feedback-based plan updates, database storage, and FastAPI input processing.
3	Monica G	Milestone 3 – routes.py Development	Implement core routes, connect Gemini functions and database functions, and manage request/response flow.
4	Monisha Anandan	Milestone 4 – Frontend Development	Design responsive HTML/CSS UI and implement Jinja2 templates for user input, results, feedback and admin view.
5	Mridula M	Milestone 5 – Deployment & Testing	Prepare local deployment, configure environment variables/dependencies, run Uvicorn, and test local application/API documentation.

Overall Code Quality Assessment

Aspect	Rating (1-5)	Comments
Code Layout & Structure	5	Modular FastAPI architecture separates routes, AI generation, database operations and templates.
Readability	4	Functions and modules have descriptive names and a clear responsibility-based structure.
Reusability	5	AI generation and database operations are encapsulated in reusable functions/modules.
Documentation / Comments	4	The project document clearly explains milestones, activities and module responsibilities; more inline code comments could be added.
Overall Score	4.5	Strong overall structure with good modularity and clear separation of responsibilities.

Project Module Summary

FitBuddy is a web-based AI fitness application that generates personalized 7-day workout plans and nutrition/recovery tips from user details such as age, weight, fitness goal and workout intensity. Users can submit feedback to refine an existing plan, while an admin view displays registered users and their original/updated plans.

Technology Stack

Backend: FastAPI and Python • AI: Google Gemini 1.5 Pro and Gemini Flash • Database: SQLite with SQLAlchemy • Frontend: HTML, CSS and Jinja2 • Server: Uvicorn

Reference to Submitted Project

The work allocation and code-quality assessment above are aligned with the milestones and activities documented in the FitBuddy project document, including model selection, core functionality development, routes.py development, frontend development, and local deployment/testing.